18 · Domain Playbooks

> **Goal:** show how the platform ideas from chapters 1-17 bend for five recurring problem shapes (recommendation, fraud, forecasting, unstructured inputs, decision-centric ML) including the operational runbook items interviewers probe.

All data in this chapter is synthetic and every system described is a generic composite; no claim is made about any real dataset or company. Each playbook ends its Conceptual Deep Dive with the operational runbook items to remember.

Table of Contents

- 18.1 Recommender Systems in Production
- 18.2 Fraud, Anomaly Detection and Adversarial Settings
- 18.3 Forecasting and Time-Series Operations
- 18.4 Vision, NLP-Classic and Document Pipelines in Production
- 18.5 Decision-Centric ML: Churn, Credit, Pricing and Uplift

18.1 Recommender Systems in Production

Conceptual Deep Dive

A production recommender is a funnel. **Candidate generation** cuts millions of items to hundreds using a two-tower model with approximate nearest-neighbour (ANN) search, co-visitation tables or popularity lists. A **ranker** (GBDT or deep model) scores candidates with richer features. A **re-ranker** applies diversity, freshness and business rules. **Serving** logs exactly what was shown, at which position, with which model version, because that exposure log is tomorrow's training data.

Training data is implicit feedback (clicks, watch time), so absence of a click is not a negative. Use negative sampling carefully, and model **position bias**: users examine top slots more, so raw click rates reward whatever was already ranked high. Log propensities or run a small randomisation slice so inverse-propensity scoring (IPS) can correct estimates (see 7.18 for off-policy evaluation).

Evaluation: offline recall@k, NDCG, calibration, catalogue coverage and diversity; online A/B tests or interleaving (cheaper in traffic for ranker comparisons), watching novelty and long-term retention rather than only click-through. Offline numbers computed on logs produced by the previous recommender are biased toward it.

Cold start: new users get popularity, context (device, locale) and onboarding signals; new items get content features and an exploration budget so they earn data.

Operations runbook: index build cadence and freshness SLO; embedding model versioning (an updated embedding model makes the old ANN index incompatible, so version index and model together and switch atomically); feature freshness; cache layers and invalidation; popularity-bias and feedback-loop control (exploration traffic, diversity guardrails); catalogue churn (items users saw vanish, so handle missing ids gracefully). **Failure modes**: feedback loops that narrow content over time, stale embeddings after a model change, and exposure bias in offline evaluation. Instrument each stage separately (candidate recall, ranker NDCG, re-ranker rule hits) so a regression can be assigned to a stage rather than to "the recommender".

Cross-Connection Scenarios

- Counterfactual logging and bandits are in 7.18; experimentation in chapter 7.
- Index and embedding versioning follow registry packaging in 4.16.
- Embedding drift monitors are in 18.4; feature freshness in chapter 2.
- Feed ranking fairness for creators links to 17.1.

Production Anti-Patterns

- Evaluating offline on logs from the same recommender without correction: the metric rewards repeating yesterday's ranking.
- No exploration traffic: new items and shifting tastes are never observed, and the loop narrows.
- Ranking with no diversity guardrails: engagement rises briefly while variety and retention decay.

Minimal Working Code

A tiny two-stage recommender with recall@k and NDCG on synthetic interactions.

```
import numpy as np

rng = np.random.default_rng(0)
U, I, G = 300, 120, 6
item_g, user_g = np.arange(I) % G, rng.integers(0, G, U)
pref = np.where(item_g[None, :] == user_g[:, None], 0.25, 0.01) * (0.5 + rng.random(I))[None, :]
inter = rng.random((U, I)) < pref
train, held = inter.copy(), -np.ones(U, int)
for u in range(U):
    items = np.flatnonzero(inter[u])
    if len(items) >= 3:
        held[u] = rng.choice(items)
        train[u, held[u]] = False

R = train.astype(float)
co = R.T @ R
norm = np.sqrt(np.diag(co)) + 1e-9
sim = co / np.outer(norm, norm)
np.fill_diagonal(sim, 0)
pop = R.sum(0) / R.sum(0).max()

def recommend(u, k=10, n_cand=30, w=0.7):
    s = R[u] @ sim
    s[R[u] > 0] = -np.inf                          # candidate generation: co-visitation similarit
y
    cands = np.argsort(-s)[:n_cand]
    rank = w * s[cands] / (s[cands].max() + 1e-9) + (1 - w) * pop[cands]   # heuristic ranker
    return cands[np.argsort(-rank)][:k]

def evaluate(rec_fn, k=10):
    hits, ndcg, n = 0, 0.0, 0
    for u in np.flatnonzero(held >= 0):
        rec = list(rec_fn(u))[:k]
        n += 1
        if held[u] in rec:
            hits += 1
            ndcg += 1 / np.log2(rec.index(held[u]) + 2)
    return hits / n, ndcg / n

pop_rec = lambda u: [i for i in np.argsort(-pop) if not train[u, i]][:10]
print("popularity recall/ndcg:", np.round(evaluate(pop_rec), 3))
print("two-stage  recall/ndcg:", np.round(evaluate(recommend), 3))
assert evaluate(recommend)[0] > evaluate(pop_rec)[0]
```

Position-bias correction: naive CTR versus an IPS-weighted estimate of item relevance.

```
import numpy as np

rng = np.random.default_rng(0)
n_items, n_sessions, slots = 20, 40000, 5
rel = rng.uniform(0.05, 0.5, n_items)                # true click probability if examined
legacy = rng.random(n_items)                         # old ranker's score, unrelated to rel
exam = lambda pos: 1.0 / (1.0 + pos)                 # examination probability by position

shown = np.zeros(n_items); clicks = np.zeros(n_items); ips = np.zeros(n_items)
for _ in range(n_sessions):
    items = rng.choice(n_items, slots, replace=False)
    items = items[np.argsort(-legacy[items])]         # high legacy score gets top positions
    for pos, i in enumerate(items):
        click = rng.random() < exam(pos) * rel[i]
        shown[i] += 1
        clicks[i] += click
        ips[i] += click / exam(pos)                   # weight by inverse examination propensity

naive, corrected = clicks / shown, ips / shown
c_naive = np.corrcoef(naive, rel)[0, 1]
c_ips = np.corrcoef(corrected, rel)[0, 1]
print(f"correlation with true relevance: naive CTR={c_naive:.2f}  IPS={c_ips:.2f}")
assert c_ips > c_naive
# In practice the examination curve is itself estimated (e.g. from a randomised slice), check current met
hods.
```

Interview Angle

Q: Design a feed recommender: stages, data, evaluation, and the top three failure modes. **A:** Candidate generation (two-tower with ANN plus popularity and fresh-item sources), a ranker trained on exposure-logged implicit feedback with position features, a re-ranker for diversity and policy, and a serving layer logging impressions, positions and versions. Evaluate offline with recall@k and NDCG on held-out interactions, then interleaving and A/B on retention. Failures: feedback loops, stale embeddings versus index, exposure bias. Trade-off: relevance versus exploration. Check: a randomised slice's metrics and index/model version parity alerts.

Q: How do you handle cold-start items and users? **A:** Items: content-based embeddings from metadata, a bounded exploration budget (for example a small traffic share, rule of thumb) with decay as data accrues. Users: popularity within context (locale, device), onboarding preferences, and fast adaptation from the first few interactions. Trade-off: exploration costs short-term engagement. Check: time-to-first-impressions for new items and day-7 retention of new users versus a holdout.

18.2 Fraud, Anomaly Detection and Adversarial Settings

Conceptual Deep Dive

Fraud problems combine **extreme imbalance** (fractions of a percent positives), **delayed and censored labels** (chargebacks arrive weeks later, many frauds are never labelled), **adaptive adversaries**, and **asymmetric costs**. Rules plus ML hybrids are normal: rules encode known patterns and regulatory blocks, models catch the rest.

Features: velocity (counts and amounts per card, device, IP in windows), device and graph features (shared devices, linked accounts), and behavioural signals. Streaming computation gives fresh velocity but costs consistency work (chapter 14); batch features are simpler but stale.

Labels come from chargebacks and analyst reviews and suffer **selection bias**: declined transactions never reveal their outcome, and reviewed cases are not a random sample. Keep an exploration slice (a small random share allowed through, within risk appetite) and a holdout to measure the true fraud rate; evaluate only on **matured cohorts** whose label window has closed.

Thresholding is cost-driven, not AUC-driven: expected loss = missed fraud loss + review cost + false-decline cost, subject to **review capacity** (precision at k when analysts can handle k cases a day). Use tiered actions (allow, step-up authentication, review, decline) and per-segment thresholds.

Adversarial drift: attackers probe thresholds, adapt within hours, and may try model extraction. Defences: rate limits, rotating or hidden features, honeypots, limited error feedback, and monitoring score distributions and attack-signature rates.

Operations runbook: a rules deploy path with shadow-mode evaluation before enforcement, explainability for analysts (10.16 style reason codes), latency SLAs, and explicit **fail-open versus fail-closed** policy per risk level when the model or feature store is unavailable. Edge cases: an attack burst shifts features in hours; label delay hides a new pattern for weeks; a threshold change overflows the review queue. Re-estimate costs periodically, since chargeback fees, analyst salaries and customer tolerance for friction all change the optimal operating point.

Cross-Connection Scenarios

- Streaming features and exactly-once concerns are in chapter 14; fallback behaviour in Module 11 section 7 (legacy).
- Threshold economics tie to 17.5; uncertainty-based routing to review as in 18.4.
- Model extraction and abuse controls are in Module 10 section 1 (legacy).
- Incident patterns (silent fallback, poisoned data) are in 11.17.

Production Anti-Patterns

- Optimising AUC instead of cost at the operating point: a better AUC can still raise expected loss under capacity limits.
- Training only on reviewed cases: the model learns the old model's selection, not fraud.
- No fail-open/closed policy per risk: an outage either blocks all customers or lets all fraud through by accident.

Minimal Working Code

Cost-sensitive threshold selection with a review-capacity constraint.

```
import numpy as np

rng = np.random.default_rng(0)
n, K = 100000, 1000                          # K = reviews per batch
fraud = rng.random(n) < 0.005
score = np.where(fraud, rng.beta(5, 2, n), rng.beta(1, 8, n))
LOSS_MISS, COST_REVIEW = 100.0, 5.0          # assumed unit costs; review catches fraud

def expected_loss(thr):
    flagged = np.flatnonzero(score >= thr)
    if len(flagged) > K:                     # capacity: keep only the top-K by score
        flagged = flagged[np.argsort(-score[flagged])][:K]
    caught = fraud[flagged].sum()
    return COST_REVIEW * len(flagged) + LOSS_MISS * (fraud.sum() - caught), len(flagged), int(caught)

for thr in (0.2, 0.4, 0.5, 0.6, 0.7, 0.8, 0.95):
    loss, nrev, caught = expected_loss(thr)
    print(f"thr={thr:.2f} reviewed={nrev:4d} caught={caught:3d}/{int(fraud.sum())} loss={loss:9.0f}&
quot;)
best = min((expected_loss(t)[0], t) for t in np.linspace(0.05, 0.99, 95))
print("best threshold:", round(best[1], 2), "loss:", round(best[0]))
assert best[0] < expected_loss(1.01)[0]      # beats flagging nothing
```

A z-score velocity baseline with a delayed-label harness that scores only matured cohorts.

```
import numpy as np

rng = np.random.default_rng(1)
n, DELAY = 40000, 45
day = rng.integers(0, 120, n)
fraud = rng.random(n) < 0.01
velocity = rng.poisson(np.where(fraud, 9, 2))   # transactions in the last hour
med = np.median(velocity)
mad = np.median(np.abs(velocity - med)) * 1.4826 or 1.0
flag = (velocity - med) / mad > 3

def matured_eval(today):
    seen = day <= today
    matured = seen & (day <= today - DELAY)     # label window closed
    tp = (flag & fraud & matured).sum()
```

```
    precision = tp / max((flag & matured).sum(), 1)
    recall = tp / max((fraud & matured).sum(), 1)
    known = fraud & matured                          # naive: unlabeled recent frauds look legitim
ate
    naive_precision = (flag & known & seen).sum() / max((flag & seen).sum(), 1)
    return round(float(precision), 2), round(float(recall), 2), round(float(naive_precision), 2)

for today in (60, 119):
    print(f"day {today}: matured precision/recall, naive precision =", matured_eval(today))
assert matured_eval(60)[2] < matured_eval(60)[0]       # the naive view understates precision
```

Interview Angle

Q: Design a real-time fraud system under a 100 ms budget with 45-day label delay. **A:** Precompute slow features (history, graph) in a feature store; compute fast velocity features in a stream; score with a compact GBDT inside the budget; tiered actions with cost-derived thresholds and a capacity cap on review. Keep rules in shadow before enforcement. For labels: exploration slice and holdout, train on matured cohorts, monitor early proxies (chargeback signals, review outcomes) while waiting. Fail-open or closed per risk tier. Trade-off: latency versus feature richness. Check: p99 latency and matured-cohort loss.

Q: How do you detect that fraudsters have adapted to your model? **A:** Watch score and feature distributions of flagged traffic, the ratio of declined to step-up-passing attempts, new clusters in graph features, and sudden precision drops in review outcomes. Probing shows as many small attempts near the threshold from linked devices. Compare against the holdout slice to see true fraud rate movement. Trade-off: sensitivity versus alert noise. Check: detection delay measured by injecting simulated attack patterns into shadow evaluation.

18.3 Forecasting and Time-Series Operations

Conceptual Deep Dive

Forecasting covers demand, load, capacity and financial series, differing in **horizon**, **granularity**, **hierarchy** (SKU, store, region, total) and **intermittency** (many zero periods). The first design decision is what the forecast drives: inventory, staffing or capacity, because that determines which error costs more.

Evaluation uses **rolling-origin backtesting**: repeatedly train on data up to an origin, forecast the next horizon, and slide forward. Random train/test splits leak the future. Metric choice matters: MAPE explodes or is undefined near zero (zero-inflated series), sMAPE is asymmetric, MASE scales errors by a naive benchmark so it is comparable across series, and WAPE (sum of absolute error over sum of actuals) is robust and business-readable. Report bias (mean signed error) separately from accuracy, because a biased forecast systematically over- or under-stocks. Prediction intervals need coverage checks, not just point error.

Baselines first: seasonal naive, ETS or ARIMA, GBDT on lag and calendar features, and global deep models. Simple methods often win on short or noisy series; complexity must beat seasonal naive in the backtest.

Pipelines: leakage through future covariates (only use covariates known at forecast time), calendar and holiday features, promotions, and hierarchical reconciliation so that aggregated and disaggregated forecasts agree (bottom-up, top-down or optimal reconciliation concepts).

Operations runbook: retraining cadence, refit versus re-tune, forecast versioning, forecast-versus-actual dashboards, drift as level shift or seasonality change, and override workflows where planners adjust forecasts, with every override logged and later scored for value added. **Forecast value** is downstream: bias costs, safety stock, stockout versus waste. Edge cases: structural break (policy or pandemic-like event) invalidates history, so down-weight or truncate it and widen intervals; zero-inflated series break MAPE; aggregates disagree with parts. Also decide who may override and how: planners often know about promotions the model cannot, which is why logged overrides, scored later, are valuable.

Cross-Connection Scenarios

- Decision costs and capacity link to 18.5 and the threshold logic in 18.2.
- Level-shift detection reuses the changepoint method in 11.18.
- Retraining triggers are in chapter 5; drift dashboards in chapter 8.
- Leakage tests belong in 13.5.

Production Anti-Patterns

- Random train/test split on a time series: future information leaks and the metric is optimistic.
- Reporting only MAPE: zeros and low volumes distort it and bias is invisible.
- Overriding forecasts without logging overrides: you cannot learn whether humans add value.

Minimal Working Code

A rolling-origin backtest comparing seasonal naive with simple exponential smoothing, scored by MASE and WAPE.

```
import numpy as np

rng = np.random.default_rng(0)
t = np.arange(260)
y = 100 + 0.1 * t + 20 * np.sin(2 * np.pi * t / 7) + rng.normal(0, 3, len(t))

def snaive(train, h, m=7):
    return np.array([train[-m + (i % m)] for i in range(h)])

def ses(train, h, alpha=0.3):
    level = train[0]
    for v in train[1:]:
        level = alpha * v + (1 - alpha) * level
    return np.full(h, level)

def backtest(y, fc, h=7, start=120, step=7, m=7):
    mase, abs_err, abs_act = [], 0.0, 0.0
    for origin in range(start, len(y) - h + 1, step):
        train, actual = y[:origin], y[origin:origin + h]
        pred = fc(train, h)
        scale = np.mean(np.abs(train[m:] - train[:-m]))      # seasonal-naive in-sample error
        mase.append(np.mean(np.abs(actual - pred)) / scale)
        abs_err += np.abs(actual - pred).sum()
        abs_act += np.abs(actual).sum()
    return round(float(np.mean(mase)), 3), round(float(abs_err / abs_act), 3)

for name, fc in [("seasonal naive", snaive), ("SES", ses)]:
    print(f"{name:15s} MASE={backtest(y, fc)[0]} WAPE={backtest(y, fc)[1]}")
assert backtest(y, snaive)[0] < backtest(y, ses)[0]
```

Prediction intervals from backtest residuals, with a coverage check on a later window.

```
import numpy as np

rng = np.random.default_rng(0)
t = np.arange(320)
y = 100 + 20 * np.sin(2 * np.pi * t / 7) + rng.normal(0, 3, len(t))
H, M = 7, 7
snaive = lambda train: np.array([train[-M + (i % M)] for i in range(H)])

def errors(origins):
    return np.array([y[o:o + H] - snaive(y[:o]) for o in origins])      # shape (origins, H)

calib = errors(range(60, 200))                                  # calibration origins
q_lo, q_hi = np.percentile(calib, [10, 90], axis=0)                  # per-horizon-step quantiles
hold_origins = range(210, len(y) - H + 1)
covered = [(y[o:o + H] >= snaive(y[:o]) + q_lo) & (y[o:o + H] <= snaive(y[:o]) + q_hi) for o in
 hold_origins]
coverage = np.mean(covered)
print(f"nominal 80% interval, empirical holdout coverage = {coverage:.3f}")
assert 0.7 < coverage < 0.9                    # near nominal when the noise level is stable
```

Interview Angle

Q: How do you backtest a forecasting model and what do you report? **A:** Rolling-origin evaluation that mimics production: expanding or sliding window, the real horizon, only covariates known at forecast time. Report WAPE and MASE against seasonal

naive, bias, interval coverage, and results by hierarchy level and segment, not one average. Trade-off: more origins give stabler estimates but cost compute. Check: the model must beat seasonal naive in most origins, with stable error across origins.

Q: Forecasts degraded after a pricing policy change. What do you do? **A:** Treat it as a structural break. Confirm with changepoint analysis against the policy date, add the policy as a covariate if it will recur, down-weight or truncate pre-change history, widen intervals, and increase monitoring frequency. Use planner overrides temporarily, logged. Trade-off: less history means higher variance. Check: post-break backtest error versus seasonal naive, and interval coverage recovering.

18.4 Vision, NLP-Classic and Document Pipelines in Production

Conceptual Deep Dive

Computer vision operations centre on data: annotation workflows with guidelines and inter-annotator agreement checks, augmentation matched to real variation, class imbalance handling, edge deployment (model size, quantisation, over-the-air updates with staged rollout and on-device rollback) and **input-quality drift** (camera, lighting, compression, firmware). Monitor image statistics (brightness, blur, resolution) because a firmware update that changes colour profile degrades accuracy without any code change.

Classic NLP (classification, NER, search ranking) has its own drift: tokeniser and vocabulary versions must travel with the model, language mix and text length shift, new slang appears, and label taxonomy changes invalidate historical labels (version the taxonomy and relabel or map). Monitor embedding drift and out-of-vocabulary rate.

OCR and document pipelines chain stages (layout, OCR, field extraction, validation), so errors compound: 95% at each of three stages is about 86% end to end (arithmetic illustration, errors assumed independent). Propagate confidence between stages, track **field-level** metrics, and put a **human review loop** on low-confidence fields, with reviewer corrections feeding retraining.

Evaluation uses per-class and per-condition slices (lighting, device, document template), confusion matrices and per-class thresholds rather than one global accuracy.

Monitoring unstructured inputs where labels arrive late or never: embedding-based drift (mean shift, domain-classifier accuracy), image or text statistics, language mix, and the rate of low-confidence predictions. This gives early warning before ground truth.

Operations runbook: input-quality gates that reject unreadable inputs, versioned preprocessing, staged device rollouts, and a review-queue SLA. Generative and LLM systems belong in chapter 19 and the LLMOps sibling project. Edge cases: camera firmware changes colour profile; taxonomy change invalidates labels; OCR errors compound across stages. Keep a golden set per condition (device, lighting, template) and re-run it on every model or preprocessing change, because aggregate accuracy hides a collapse on one condition.

Cross-Connection Scenarios

- Drift monitoring mechanics are in chapter 8; embedding versioning in 18.1.
- Confidence routing is the same abstention idea as 17.5.
- Staged rollouts and rollback are in chapter 7; edge packaging in 4.16.
- Slice-aware testing is in 13.5.

Production Anti-Patterns

- Monitoring only accuracy on a static test set: field conditions drift while the test set never changes.
- No human review loop for low-confidence outputs: errors flow downstream and no corrected labels are collected.
- Ignoring input-quality gates: garbage inputs yield confident garbage outputs.

Minimal Working Code

An embedding-drift monitor using mean shift and a domain-classifier accuracy proxy.

```
import numpy as np

rng = np.random.default_rng(0)
d = 16

def drift_report(ref, cur):
```

```
    pooled = np.sqrt((ref.var(0) + cur.var(0)) / 2)
    mean_shift = float(np.linalg.norm((cur.mean(0) - ref.mean(0)) / pooled) / np.sqrt(d))
    # domain classifier proxy: project on the mean-difference direction, fit on half, test on the other h
alf
    h = min(len(ref), len(cur)) // 2
    w = cur[:h].mean(0) - ref[:h].mean(0)
    thr = (cur[:h].mean(0) + ref[:h].mean(0)) @ w / 2
    acc = 0.5 * ((ref[h:2 * h] @ w < thr).mean() + (cur[h:2 * h] @ w >= thr).mean())
    return round(mean_shift, 3), round(float(acc), 3)

ref = rng.normal(0, 1, (2000, d))
same = rng.normal(0, 1, (2000, d))
shifted = rng.normal(0, 1, (2000, d)) + np.r_[0.4, 0.4, 0.4, np.zeros(d - 3)]
print("no drift   (mean shift, domain-clf acc):", drift_report(ref, same))
print("with drift (mean shift, domain-clf acc):", drift_report(ref, shifted))
assert drift_report(ref, shifted)[1] > drift_report(ref, same)[1] + 0.1     # accuracy near 0.5 means
no drift
```

Confidence routing for a multi-stage document pipeline, measuring review rate versus residual error.

```
import random

random.seed(0)
fields = []
for _ in range(5000):
    ocr, parse = random.betavariate(8, 1.5), random.betavariate(8, 1.5)
    conf = ocr * parse                              # confidence propagates multiplicatively
    fields.append((conf, random.random() < conf ** 0.7))  # synthetic: higher confidence, more often c
orrect

def route(fields, threshold):
    auto = [ok for conf, ok in fields if conf >= threshold]
    review_rate = 1 - len(auto) / len(fields)
    residual_error = 1 - sum(auto) / len(auto) if auto else 0.0
    return round(review_rate, 3), round(residual_error, 3)

for thr in (0.3, 0.5, 0.7, 0.8):
    review, err = route(fields, thr)
    print(f"threshold={thr}: review rate={review}, error among auto-accepted={err}")
assert route(fields, 0.8)[1] < route(fields, 0.3)[1]
```

Interview Angle

Q: How do you monitor an image classifier whose labels arrive weeks later? **A:** Use label-free signals: image statistics (brightness, blur, resolution), embedding drift versus a reference window, prediction-distribution and low-confidence rates, per-device and per-site slices. Keep a continuously labelled sample (active sampling of uncertain and random cases) to estimate accuracy quickly. Trade-off: label cost versus detection speed. Check: drift alerts should precede measured accuracy drops in backtests of past incidents, with a false-alert rate the team can handle.

Q: Design the human-review loop for a document extraction pipeline. **A:** Route fields by calibrated confidence and business criticality (amounts always get a stricter threshold). Reviewers see the image crop, the extracted value and alternatives, and their corrections are stored with the model version as training data. Set a queue SLA and sample auto-accepted fields for audit. Trade-off: review cost versus error rate. Check: error among auto-accepted fields, review rate, reviewer disagreement and throughput.

18.5 Decision-Centric ML: Churn, Credit, Pricing and Uplift

Conceptual Deep Dive

The goal is rarely a prediction; it is a **decision** with a cost matrix, a capacity limit or a budget. Frame it as the expected value of acting: probability of the outcome times benefit of the action, minus the cost of the action, ranked and cut at budget.

Churn illustrates the trap. Predicting who will leave is not the same as predicting who will stay because of an intervention: high-risk customers include "lost causes" who leave regardless and "sure things" who would have stayed. Define the churn window and the retention action's cost and effect first.

Uplift modelling estimates the **treatment effect** per customer, not the risk. Approaches: two-model (separate models for treated and control, subtract), meta-learners and uplift trees. Evaluate with **Qini or uplift curves** (cumulative incremental gain when targeting top-ranked customers) on **randomised** data; observational data needs adjustment for confounding and still can mislead. Keep a randomised holdout to measure true effect continuously.

Credit scoring needs monotonic constraints (more income should not lower the score without reason), scorecards versus GBDT trade-offs (interpretability against accuracy), calibration, and reason codes (17.3). **Reject inference** is the selection problem: outcomes are observed only for approved applicants, so a model trained on them is biased about the rejected region; remedies include exploration within risk appetite, bounded randomised approvals, and augmentation methods used with caution.

Pricing and elasticity: price changes feed back into demand, experiments contaminate each other (shared inventory, cross-product substitution), and regulation may constrain differential pricing (verify rules). Use guardrails and switchback or geo-level designs.

Operations runbook: governance of thresholds (who may change a cutoff), monitoring decision rates and realised value rather than only scores, champion/challenger on decisions (A/B the policy, not the AUC), and a persistent control group. Edge cases: risk targeting spends on customers who would have stayed; selection bias in credit outcomes; price tests contaminating each other.

Cross-Connection Scenarios

- Randomisation and off-policy ideas are in 7.18; fairness of decisions in 17.1.
- Cost-based thresholds mirror 18.2; calibrated probabilities from 17.5 feed expected value.
- Forecast-driven decisions (stock, capacity) are in 18.3.
- Reason codes and credit governance in 17.3 and 4.15.

Production Anti-Patterns

- Optimising model metric not decision value: AUC gains that never change who gets treated create nothing.
- Using observational data for uplift without adjustment: confounded treatment assignment yields invented effects.
- No holdout to measure true treatment effect: the programme's value is assumed, never verified.

Minimal Working Code

Expected-value targeting: risk ranking versus uplift ranking on simulated randomised data, with a Qini-style gain table.

```
import numpy as np

rng = np.random.default_rng(0)
n = 40000
x0, x1 = rng.normal(size=n), rng.normal(size=n)
sig = lambda z: 1 / (1 + np.exp(-z))
p0 = sig(1.5 * x0 - 0.5)                          # baseline churn risk, driven by x0
tau = -0.25 * sig(3 * x1)                          # treatment lowers churn; effect driven by x1
treated = rng.random(n) < 0.5                      # randomised assignment
churn = rng.random(n) < np.clip(p0 + treated * tau, 0, 1)

A = np.c_[np.ones(n), x0, x1]
tr, te = np.arange(n) < n // 2, np.arange(n) >= n // 2
fit = lambda m: np.linalg.lstsq(A[m], churn[m].astype(float), rcond=None)[0]
b_c, b_t = fit(tr & ~treated), fit(tr & treated)   # two-model uplift (linear probability)
risk, uplift = A @ b_c, A @ b_c - A @ b_t          # uplift = predicted churn reduction

def qini(score, fracs=(0.1, 0.2, 0.3, 0.5, 1.0)):
    idx = np.flatnonzero(te)
    order = idx[np.argsort(-score[idx])]
    out = []
    for f in fracs:
        top = order[: int(f * len(order))]
        t, c = top[treated[top]], top[~treated[top]]
        out.append(round(float((~churn[t]).sum() - (~churn[c]).sum() * len(t) / max(len(c), 1)), 0))
```

```
    return out

print("customers retained by targeting top 10/20/30/50/100%:")
print("risk ranking  :", qini(risk))
print("uplift ranking:", qini(uplift))
assert qini(uplift)[1] > qini(risk)[1]
```

Budget-constrained targeting by expected incremental profit, with sensitivity to treatment cost.

```
import numpy as np

rng = np.random.default_rng(0)
n, BUDGET, VALUE = 10000, 1500, 120.0          # assumed value of a retained customer
effect = rng.beta(1.2, 8, n)                   # estimated churn reduction per customer in [0, 1]

def plan(cost):
    profit = effect * VALUE - cost             # expected incremental profit per treated customer
    order = np.argsort(-profit)[:BUDGET]       # capacity limit
    chosen = order[profit[order] > 0]          # never treat at negative expected profit
    return len(chosen), round(float(profit[chosen].sum()), 0)

for cost in (5, 10, 20, 30, 40):
    count, total = plan(cost)
    print(f"treatment cost={cost:3d}: treat {count:4d} customers, expected profit={total}")
assert plan(5)[1] > plan(40)[1] and plan(40)[0] < plan(5)[0]
```

Interview Angle

Q: Marketing wants a churn model. How do you make sure it creates value? **A:** Start from the decision: what action, at what cost, with what effect. Run a randomised pilot to measure treatment effect, then rank by expected incremental profit (uplift times value minus cost), not by churn risk. Set budget and thresholds with finance, monitor realised value with a persistent holdout, and compare policies via champion/challenger on decisions. Trade-off: randomisation sacrifices short-term savings. Check: incremental retained customers per cost versus control.

Q: How do you evaluate a credit model when you only observe outcomes for approved applicants? **A:** Acknowledge the selection bias. Options: a small, risk-bounded exploration set of approvals outside the cutoff, reject inference methods applied cautiously with sensitivity analysis, using earlier policy variation or natural experiments, and tracking approved-population stability. Report evaluation on the approved set as a lower-confidence estimate. Trade-off: exploration costs real losses. Check: performance on the exploration slice versus predicted risk, and calibration by score band.